GenAI Bootcamp — Coding Assignment

Objective

Assess the candidate's practical skills, understanding, and innovative application of Generative AI frameworks and tools through a small project demonstration

Overview

Build a small **agentic research assistant** in Python that answers compound questions about a provided local corpus of text files. This demo project needs to showcase your skills and knowledge in GenAI solutions, with a focus on Python programming. The project should demonstrate your ability to integrate and utilize GenAI services and APIs.

Window: November 11 – November 13, 2026 (3 calendar days).

Expected effort: ~1 full focused day of work puts you at ~90% completion. The extra days are for scheduling flexibility, iteration, and polish.

Scope of Assignment

Domain: pharmacology. The provided corpus is a curated set of ~20 .md / .txt files derived from **public FDA drug labels (DailyMed)** and **consumer medication information** (e.g., MedlinePlus-style monographs). Expect sections like indications, dosage, contraindications, interactions, and mechanism of action. A GLOSSARY.md is included so terminology is not a barrier. **This is a technical exercise, not medical advice.**

©2026 Syntax Systems. All Rights Reserved.
Proprietary and Confidential. Reproduction or distribution is strictly prohibited without express written consent.

Given a user question and this corpus, your agent must:

1. **Plan :** decompose the question into sub-questions or steps.
2. **Retrieve :** fetch relevant passages from the corpus using in-memory similarity.
3. **Use tools :** at least two local tools. Tool choice must be driven by the agent, not hardcoded.
4. **Review :** a critic step verifies grounding and flags unsupported or clinically ambiguous claims before the final answer is emitted.
5. **Answer** : produce a cited response referencing the source file(s) used, with an explicit disclaimer that the output is not medical advice.

The corpus and a set of 5 evaluation questions (mix of single-document lookups and multi-drug / multi-hop reasoning) are provided. We will also provide a LiteLLM key to call OpenAI models (you can still use your own api keys and your prefered LLM provider)

Technical Requirements

- Python 3.11+, dependencies pinned in pyproject.toml or requirements.txt.
- Agent orchestration framework is your choice (LangChain, LangGraph, PydanticAI, your own loop).
- You will be assessed on the quality of your code and user experience

Interface (mandatory)

- A Dockerfile at the repo root. docker build -t agent . && docker run -p 8000:8000 --env-file .env agent must start your API. The container is self-contained.
- We run your container with our own values.
- GET /health → 200.

©2026 Syntax Systems. All Rights Reserved.
Proprietary and Confidential. Reproduction or distribution is strictly prohibited without express written consent.

- POST /ask with {"question": "..."} → 200
- A minimal frontend (any technology) that calls POST /ask and shows the answer. You may add anything beyond this.

Deliverables

- Public Git repository link (preferably Gitlab).

Presentation

Prepare a 10 minute presentation to showcase your project. This presentation can be in any format you prefer; it does not necessarily have to be a PowerPoint presentation. A code walkthrough is also acceptable and can be an effective way to demonstrate the intricacies and functionality of your project. Highlight:

- The problem addressed.
- The GenAI frameworks and tools used and the rationale behind their selection.
- The architecture and workflow of the application.
- Key challenges faced, particularly in terms of integration and advanced functionality, and how they were overcome.

Evaluation Criteria

Skills	Description
Technical Competence	Depth and sophistication of the implementation.
Innovation and Creativity	Originality and innovative use of advanced GenAI tools.
Problem-Solving	Effectiveness in tackling complex problems with advanced solutions.

©2026 Syntax Systems. All Rights Reserved.
Proprietary and Confidential. Reproduction or distribution is strictly prohibited without express written consent.

Evaluation Questions

The corpus and 4 public evaluation questions are provided. Hidden questions are held back by the reviewers.

Questions	Expected source files
What are the main contraindications for warfarin?	**dailymed-warfarin.md**
Compare the contraindications of warfarin and apixaban. What conditions are shared and which are unique to each drug?	**dailymed-warfarin.md, dailymed-apixaban.md**
A patient currently on warfarin is prescribed sertraline. What is the risk of bleeding and what monitoring is recommended?	**dailymed-warfarin.md, dailymed-sertraline.md**
A patient with an eGFR of 25 mL/min is being considered for metformin. Is metformin contraindicated at this level of renal function, and what is the threshold	**dailymed-metformin.md**

AI Assistance — Policy & Transparency

AI assistance (Copilot, Claude, Cursor, ChatGPT, etc.) is permitted and encouraged. This mirrors how we actually work. However:

- **You are accountable for every line of code in the repo**, AI-generated or not. In the interview we may ask you to walk through any file and explain design decisions.
- Push your **AI coding configuration** (skills, mcp config file, etc.) to your repo

©2026 Syntax Systems. All Rights Reserved.
Proprietary and Confidential. Reproduction or distribution is strictly prohibited without express written consent.

Submission

Email the repo link (public) and a short summary of your approach to the recruitment contact by **23:59 (Casablanca time) on November 13**. We assess the last commit pushed before the deadline. Ensure that the demo is fully operational, with a focus on seamless setup and execution by the interviewers.

We will provide

- A — A LiteLLM proxy key, base URL and the allowed model names and a code snippet showing how to use the key (see below + .env).
- B — The corpus of ~20 markdown files, including GLOSSARY.md.

LLM Access

Your key works with any OpenAI-compatible SDK. The .env file in this package contains the credentials. Read these variables from the environment. When we grade your submission we inject our own .env.

```
import os


from dotenv import load_dotenv

from openai import OpenAI


# Load variables from .env

load_dotenv()


# OpenAI() reads OPENAI_API_KEY and, if configured, OPENAI_BASE_URL

client = OpenAI()
```

©2026 Syntax Systems. All Rights Reserved.
Proprietary and Confidential. Reproduction or distribution is strictly prohibited without express written consent.

```
# Create an embedding
emb = client.embeddings.create(
    model=os.getenv("OPENAI_EMBEDDING_MODEL", "text-embedding-3-small"),
    input=["warfarin"],
)

# Create a chat completion
chat = client.chat.completions.create(
    model=os.getenv("OPENAI_MODEL", "your-chat-model"),
    messages=[
        {
            "role": "user",
            "content": "In one sentence, what is a contraindication?",
        }
    ],
)

print(chat.choices[0].message.content)
```

©2026 Syntax Systems. All Rights Reserved.
Proprietary and Confidential. Reproduction or distribution is strictly prohibited without express written consent.